Systemarchitektur – CameraCar (v2.0 – Entwurf)

Datenflüsse, Threads, Verantwortlichkeiten und Dateninhalte

Ziel
- Klare Verantwortlichkeiten
- Schlanke Datenpakete zwischen Threads
- Reproduzierbare Datensätze (Bild + Metadaten)
- Fokus-Logging auf relevante Fahrereignisse
- Einfache Erweiterbarkeit (z. B. ML, neue Sensoren)

Thread 1 – Kamera-Sensor (~30 Hz)

CameraSensor
- Liest Kamera-Frames
- Bildvorverarbeitung
  - Crop
  - Farbfilter (HSV/ROI)
  - Blur / Morph / Canny
- Zeitstempel (System + Monotonic)
- Erstellt CameraFrame

Verwendet
- Camera (Abstraktion: picamera2 / opencv)
- Config (Kameraparameter)

schreibt → **CameraFrame**
- raw_image (np.ndarray)
- processed_image (np.ndarray)
- processing_params (dict/dataclass)
- timestamp (time.time_ns)
- monotonic_ns (time.monotonic_ns)
- frame_id (int)

≈ 30 Hz

Lädt Konfiguration (bei Start, optional zyklisch)

Zentraler Speicher „CameraCar“

Thread-safe Shared State (Aktueller Zustand)

Aktuelle Datenpakete
- camera_frame (CameraFrame)
- driving_state (DrivingState)
- vehicle_state (VehicleState)

liest
- CameraFrame
- DrivingState

Thread 2 – DriveController (~20 Hz)

CameraMode / DriveController
- Liest aktuelles CameraFrame
- Bildverarbeitung (Lanes, ROI, etc.)
- Berechnet Fahrparameter
  - Lenkwinkel (PID)
  - Geschwindigkeit
  - Lost-Line Erkennung
- Sendet Befehle an Fahrzeug
- Erstellt DrivingState

Verwendet
- Fahralgorithmen (PID, Lanes, ...)
- Config (Regelparameter)
- BaseCar (Hardware-Abstraktion)

schreibt → **DrivingState**
- steering_angle (float)
- speed (float)
- lane_detected (bool)
- lane_quality (float)
- timestamp (time.time_ns)
- debug_info (optional)

≈ 20 Hz

sendet Befehle

Thread 3 – Dashboard & Telemetrie (~15–30 Hz)

Dash / dash_live.py
- Liest aktuelles CameraFrame
- Liest aktuelles DrivingState
- Visualisierung (Live-Bild, Overlay)
- Status (Winkel, Speed, Lanes, ...)
- Anzeige von Ereignissen (z. B. Line Lost)

Verwendet
- Dash (Plotly/Stream)
- Config (Darstellung)

stellt Parameter bereit

Konfiguration – ConfigReader

- JSON-Konfiguration (z. B. config.json)
- Kamera-Parameter
- Regelparameter
- Dashboard-Parameter
- Speicherpfade
- ...

Abstraktion

Fahrzeug / Aktuatoren (Hardware)

- Lenkung (Lenkwinkel)
- Antrieb (Geschwindigkeit)

Thread 4 – Recorder (Dataset) (~10 Hz)

liest
- CameraFrame
- DrivingState

SaveData_4_AI
- Liest CameraFrame und DrivingState
- Speichert Bilder auf SSD (.jpg)
- Schreibt Metadaten in CSV
- Verlinkt Bild + CSV (frame_id / timestamp)

Verwendet
- Config (Speicherpfade, Format)
- Ereignis-Trigger (nur relevante Frames)

Legende
- Datenfluss (Kamera → Speicher)
- Datenfluss (Verarbeitung → Speicher)
- Datenfluss (Speicher → Dashboard)
- Datenfluss (Speicher → Recorder)
- Konfiguration / Steuerung
- Datenpaket (Beispiel)

Beispielhafte Ereignisse (Logging / Events)

- LANE_LOST – Linie nicht mehr erkannt (mit Bild- und CSV-Referenz)
- LANE_REGAINED – Linie wieder gefunden
- HIGH_CROSS_TRACK_ERROR – großer Spurfehler
- OBSTACLE_DETECTED – (falls weitere Sensoren vorhanden)
- SYSTEM_ERROR – Kamera, Hardware, etc.
- MODE_CHANGED – z. B. Manuell → Autonom
- SHUTDOWN – kontrolliertes Beenden